Plantilla de Diapositivas Pu++

Por: Mariana, Wendy, Emi

Club de Programación Competitiva Pu++

13 de marzo de 2026

Contenido

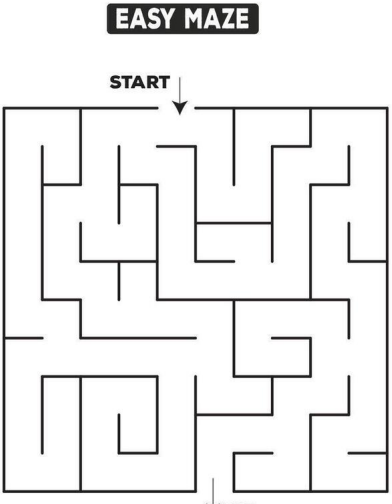
- Introducción

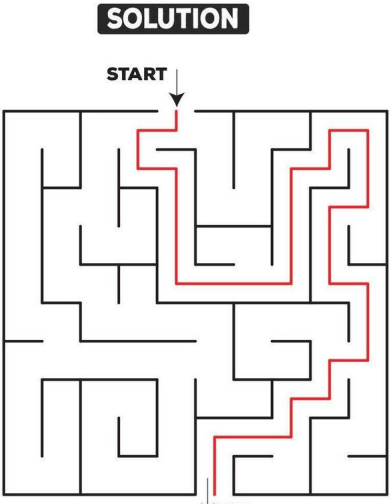
Introducción

Muchos problemas en computación utilizan algoritmos

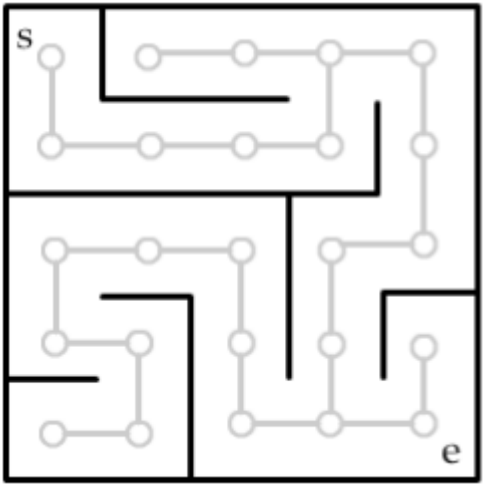
- Resolver un laberinto.
- Ver si podemos llegar de un punto A desde cualquier lugar.
- Ver si existe un ciclo en una gráfica.
- Ver si una gráfica es bipartita.

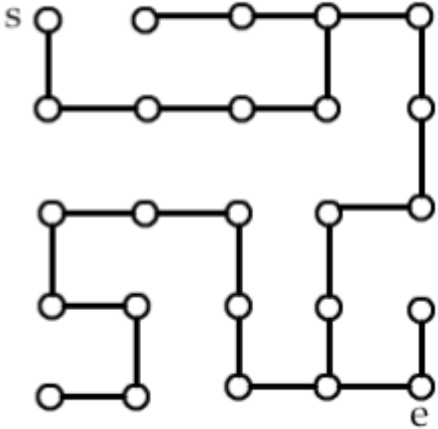
Laberinto





Laberinto en gráficas





Estructuras de datos

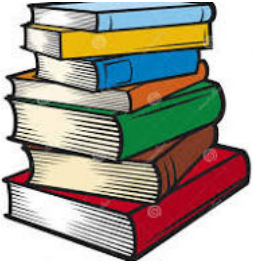
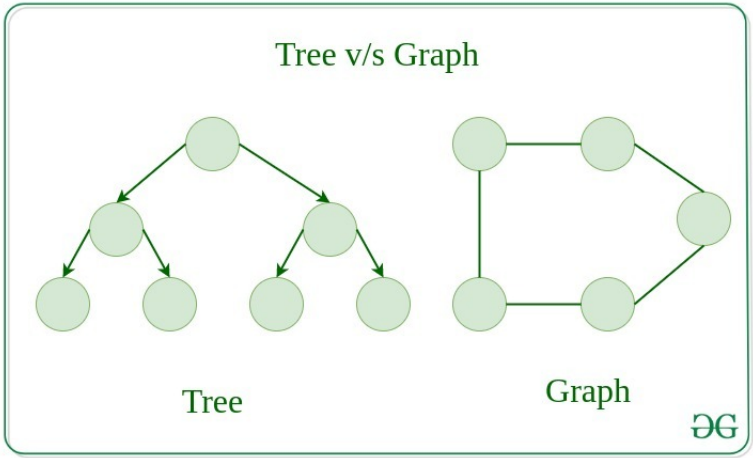


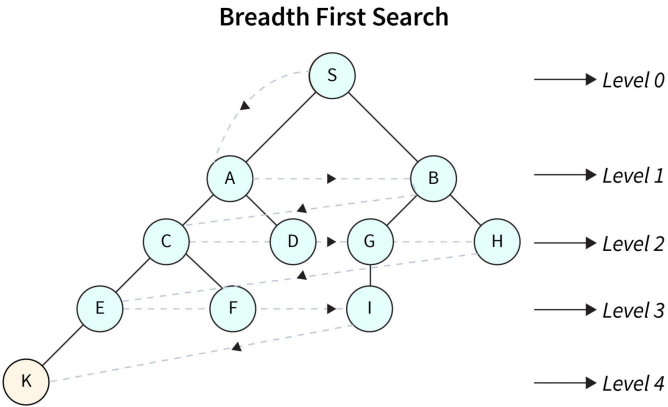
Figura: Pilas

Figura: Colas

Empecemos por árboles



BFS


Breadth First Search
S
A
B
C
D
G
H
E
F
I
K
Level 0
Level 1
Level 2
Level 3
Level 4

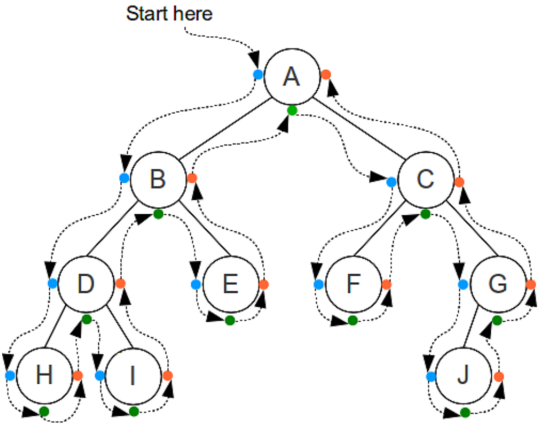
BFS code

```cpp
// --- Imports y todo lo que necesitamos ---

queue<int> q;
bool visited[N];
int distance[N];
vector<int> adj[N];

void dfs(int x) {
  visited[x] = true;
  distance[x] = 0;
  q.push(x);
  while (!q.empty()) {
    int s = q.front();
    q.pop();
    // process node s
    for (auto u : adj[s]) {
      if (visited[u])
        continue;
      visited[u] = true;
      distance[u] = distance[s] + 1;
      q.push[u];
    }
  }
}

// --- Un main ---
```

BFS Complejidad

- $O(V + E)$ con listas de adyacencias.
- $O(V^2)$

DFS



Pre-Order	ABDHIECFGJ
In-Order	HDIBEAFCJG
Post-Order	HIDEBFJGCA

DFS code

```
// --- Imports y todo lo que necesitamos ---

vector<int> adj[N];
bool visited[N];

void dfs(int s) {
  if (visited[s])
    return;
  // process node s
  for (auto u : adj[s]) {
    dfs[u];
  }
}

// --- Un main ---
```

DFS Complejidad

- $O(V + E)$ con listas de adyacencias.
- $O(V^2)$

Ejemplo de unos problemas

- **Tree Distances I**

  Judge: CSES

- **Party**

  Judge: CodeForces

Easy

Easy

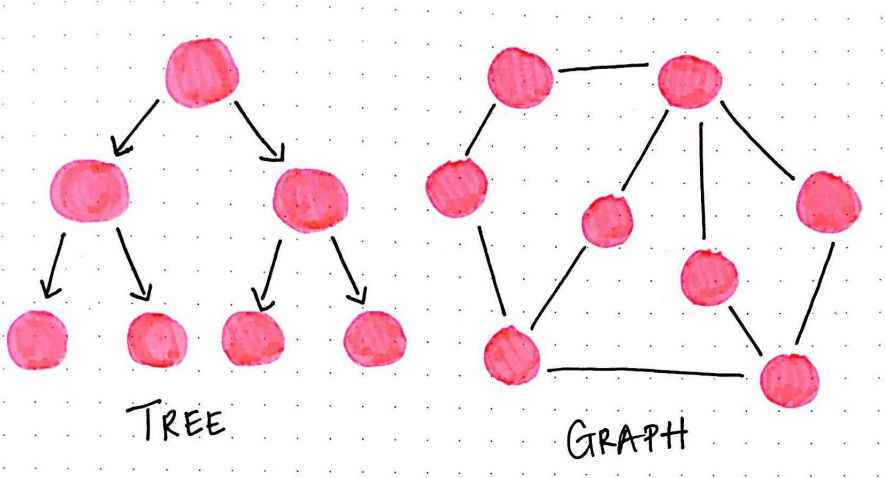
Hint

DFS (lol)

Solución

```cpp
#include<bits/stdc++.h>
using namespace std;

vector<int> v[2001];
int c = 1;

void dfs(int x, int d) {
  c = max(c,d);
  for(int i = 0; i < v[x].size(); i++) {
    dfs(v[x][i] , d + 1);
  }
}

int main() {
  int t;

  cin>>t;
  for(int i = 1; i <= t; i++) {
    int x;
    cin >> x;
    if(x == -1)
      continue;
    v[x].push_back(i);
  }
  for(int i = 1; i <= t; i++) {
    dfs(i,1);
  }
  cout << c << endl;
  // this is for academic purposes and is not part of the (bad) solution
  for (int i = 1; i <= t; i++) {
    printf("%d): ", i);
    for (int n : v[i]) {
      printf("%d\t", n);
    }
    printf("\n");
  }
}
```

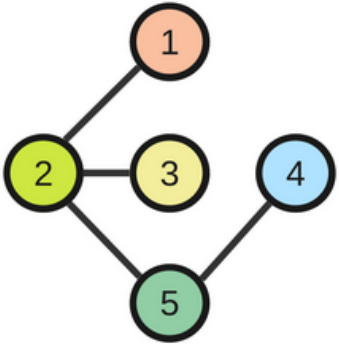
Ahora sí, gráficas

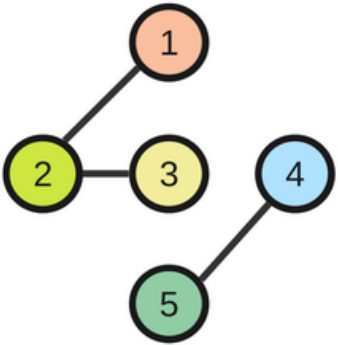


Conexidad

- Una gráfica es conexa si existe un camino entre cualesquiera par de vértices.
- Podemos ver si una gráfica es conexa si nos paramos en un vértice e intentamos llegar a cualquier nodo.

Conexidad





Otros problemas más

- **The Lake**

  Judge: CodeForces

  Easy

- **Maze**

  Judge: CodeForces

  Easy

Hint

Cada lago puede verse como una componente conexa...

Solución con DFS

```cpp
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};
int n,m;

ll dfs(ll x, ll y, vvll& grid, vvll& visited){
    visited[x][y] = 1;
    ll c = grid[x][y];

    fori(i,0,4) {
        ll nx = x+ dx[i];
        ll ny = y + dy[i];

        if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
        ↪  !visited[nx][ny] && grid[nx][ny] > 0) {
            visited[nx][ny] = true;
            c += dfs(nx,ny,grid, visited);
        }
    }
    return c;
}

void solve() {
    cin >> n >> m;

    vvll grid(n,vll(m));
    fori(i,0,n)
        fori(j,0,m)
            cin >> grid[i][j];


    vvll visited(n,vll(m, 0));
    ll maxi = 0;
    fori(i,0,n)
        fori(j,0,m)
            if(grid[i][j]>0 && visited[i][j] != 1)
                maxi = max(maxi, dfs(i,j, grid, visited));

    cout << maxi << endl;
}
```

Solución con BFS

```
ll bfs(ll x, ll y, vvll& grid, vvll& visited){
    queue<ll> qx, qy;
    visited[x][y] = 1;

    ll c = 0;
    qx.push(x);
    qy.push(y);

    while (!qx.empty()){
        ll cx = qx.front();
        ll cy = qy.front();

        c += grid[cx][cy];

        qx.pop();
        qy.pop();

        fori(i,0,4) {
            ll nx = cx + dx[i];
            ll ny = cy + dy[i];
            if (nx >= 0 && nx < n && ny >= 0 && ny < m &&
            ↪ !visited[nx][ny] && grid[nx][ny] > 0) {
                visited[nx][ny] = true;
                qx.push(nx);
                qy.push(ny);
            }
        }
    }
    return c;
}
```

```
void solve() {

    cin >> n >> m;

    vvll grid(n,vll(m));
    fori(i,0,n)
        fori(j,0,m)
            cin >> grid[i][j];


    vvll visited(n,vll(m, 0));
    ll maxi = 0;
    fori(i,0,n)
        fori(j,0,m)
            if(grid[i][j]>0 && visited[i][j] != 1)
                maxi = max(maxi, bfs(i,j, grid, visited));

    cout << maxi << endl;
}
```

Hint

Hay vértices que al quitarlos no generan más componentes conexas...

Solución con DFS

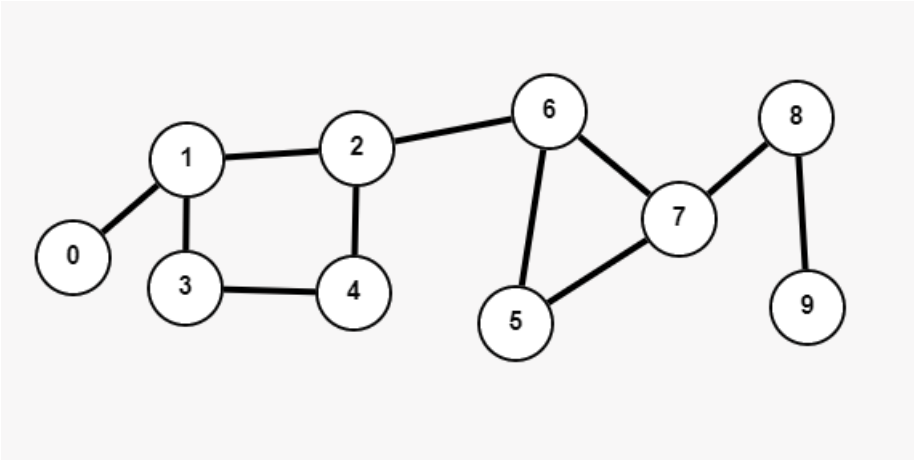
```cpp
int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

ll n, m, walls;

void dfs(ll x, ll y, vector<vector<char> >& grid, vvll & visited) {
    visited[x][y] = 1;

    fori(i,0,4) {
        ll nx = x + dx[i];
        ll ny = y + dy[i];

        if (nx >= 0 && nx < n && ny >= 0 && ny < m && grid[nx][ny] == '.') {
            if (!visited[nx][ny])
                dfs(nx, ny, grid, visited);
        }
    }

    if(walls > 0) {
        grid[x][y] = 'X';
        walls--;
    };

    return;
}
```

```cpp
void solve(){
    cin >> n >> m >> walls;
    vector<vector<char> > grid(n,vector<char>(m,'a'));

    ll startx = -1, starty;

    fori(i,0,n)
        fori(j,0,m) {
            cin >> grid[i][j];
            if(startx == -1)
                if(grid[i][j] == '.')
                    startx = i, starty = j;
        }

    vvll visited(n,vll(m,0));
    dfs(startx, starty, grid, visited);

    fori(i,0,n) {
        fori(j,0,m)
            cout << grid[i][j];
        cout << endl;
    }

}
```

Detección de ciclos

- Una gráfica contiene un ciclo si mientras la recorremos encontramos un nodo tal que un vecino suyo (que no sea del que vino) ya había sido visitado.

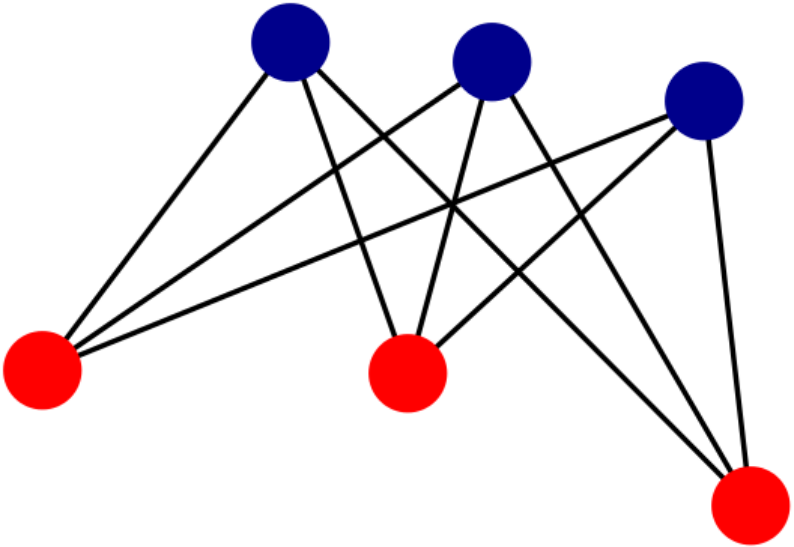
Detección de ciclos



¿Es bipartita?

- Una gráfica es bipartita si sus nodos pueden ser coloreados usando dos colores tales que ningún par de vértices adyacentes tienen el mismo color.
- ¿Cómo?

¿Es bipartita?

- Elegir dos colores X y Y.
- Colorear el nodo inicial de X, todos sus vecinos de Y, sus vecinos de sus vecinos de X, etc.
- Si en algún punto del recorrido vemos dos vértices adjacentes del mismo color, no es bipartita, en otro caso, sí.

¿Es bipartita?